SOLID Principles

Those 5 foundational guidelines for writing clean, maintainable and scalable software. \

Introduced by Robert C. Martin.

1. **S - Single Responsibility Principle (SRP)**
   A class should have **one**, and only one, reason to **change**. It should do **exactly one job**.

   ## Bad Example

   ```
   classReport {
   voidgenerateReport() { }

   voidsaveToDatabase() { }

   voidsendEmail() { }
   }
   ```

   This class handles:

   - Report generation
   - Database storage
   - Email sending

   If email logic changes, the class changes.

   ## Good Example

   ```
   classReportGenerator {
   voidgenerateReport() { }
   }

   classReportRepository {
   voidsave() { }
   }

   classEmailService {
   ```

```
voidsendEmail() { }
}
```

Each class has **one responsibility**.

2. **O - Open/Closed Principle**

   Software should be **open for extension** but **closed for modification**.

   This allows adding new functions **without modification** of existing code.

Bad Example

```
classPayment {
voidpay(Stringtype) {
if(type.equals("CreditCard")) {
// payment
        }
elseif(type.equals("UPI")) {
// payment
        }
    }
}
```

Adding a new payment method requires modifying existing code.

Good Example

```
interface PaymentMethod {
void pay();
}

class CreditCardPayment implements PaymentMethod {
public void pay() {
System.out.println("Credit Card Payment");
    }
}

class UpiPayment implements PaymentMethod {
public void pay() {
System.out.println("UPI Payment");
    }
}
```

New payment methods can be added without changing existing code.

3. L - **Liskov Substitution Principle (LSP)**

   A subclass should be able to replace its parent class without affecting program correctness.

   ## Bad Example

```
class Bird {
void fly() { }
}

class Penguin extends Bird {
void fly() {
throw new UnsupportedOperationException();
    }
}
```

   Penguins can't fly, so substituting Penguin for Bird breaks behavior.

   ## Good Example

```
class Bird { }

interface Flyable {
void fly();
}

class Sparrow extends Bird implements Flyable {
public void fly() { }
}

class Penguin extends Bird { } // it don't implements the Flyable coz penguin
can't fly.
```

   Now only flying birds implement `Flyable`.

4. **Interface Segregation Principle (ISP)**

   Clients should **not be forced** to implement methods they **don't use**.

   ## Bad Example

```
interface Worker {
void work();
void eat();
}
```

Robot doesn't eat.

```
class Robot implements Worker {
public void work() { }

public void eat() {
throw new UnsupportedOperationException();
    }
}
```

Good Example

```
interface Workable {
void work();
}

interface Eatable {
void eat();
}

class Human implements Workable,Eatable {
public void work() { }
public void eat() { }
}

class Robot implements Workable {
public void work() { }
}
```

Each class implements only what it needs.

[Design Patterns]